EFICIENCIA Y PERIODO DE LOS ALGORITMOS SHELLSORT Y QUICKSORT

Efficiency and Time Complexity of the Shellsort and Quicksort Algorithms

Adrian Mamani Quispe

Estudiante de la escuela de Ingeniería en Informática y Sistemas. Universidad Nacional Jorge Basadre Grohmann. E-mail: almamaniqu@unjbg.edu.pe

Docente adscrito al Departamento Académico de la ESIS en la UNJBG. E-mail: hmbarrazav@unjbg.edu.pe

RESUMEN

La historia de los algoritmos se remonta a la era Babilónica donde se empleaban tablas de arcilla para realizar cálculos y fórmulas matemáticas; además, de su posible surgimiento, la evolución que ha llevado es muy significativa, explicando diversos fenómenos afianzados a los seres bióticos como abióticos. Actualmente, son más frecuentes tanto como antiguos en su empleabilidad, tales como Quicksort y Shellsort y entre otras más; cabe recalcar que aun “viejos” no desmerita su aplicación en ramas como la pedagogía en educación, la variación genética en medicina o en sistemas poco y muy complejos. En este documento mostraremos datos relacionados con métricas de tiempos que evidenciarían el paralelismo entre métodos o computación basada en algoritmos de ordenación logarítmica.

Palabras clave: *Educación, medicina, computación, método, Shellsort, Quicksort, algoritmo, ordenación, empleabilidad, complejos, logarítmica.*

ABSTRACT

The history of algorithms dates back to the Babylonian era, when clay tablets were used to perform mathematical calculations and formulate equations. Beyond their origins, their subsequent evolution has been highly significant, providing explanations for various phenomena—both biological and non-biological. Today, algorithms such as

Quicksort and Shellsort remain widely used, despite their age; the fact that they are "old" does not diminish their value in fields ranging from pedagogy and medical genetics to the analysis of systems of varying levels of complexity. This document presents data on time metrics that illustrate the parallels between different methods—specifically, computational approaches based on logarithmic sorting algorithms.

Keywords: *Education, medicine, computing, method, Shellsort, Quicksort, algorithm, sorting, applicability, complex systems, logarithmic.*

INTRODUCCION

La evolución algorítmica fue afianzado con el tiempo histórico y los diversos ensayos y errores. En los orígenes del famoso “teorema de Pitágoras”, se logra evidenciar que hace 3700 años atrás aproximadamente, hubo vestigios de cálculos lógicos y aritméticos (Ciencia LR, 2024). Este informe compara los tiempos de ejecución de Shellsort y Quicksort al ordenar arreglos de distintos tamaños.

DESARROLLO

A mi opinión, las investigaciones tecnológicas modernas han aportan conocimientos valiosos acerca de la multifuncionalidad, los límites, la visión y la misión explícita que los subprogramas tienen y que los programadores han sabido manejar, resultando en herramientas como la IA y la construcción de esquemas más conscientes como los Chatbots; el desarrollo de software seguro y la planificación de módulos duraderos con el tiempo.

Creo que la conectividad de los diversos sensores, motores e instrucciones se efectúa mediante los programas principales, funciones como ramas secundarias y fundamentales. También en el mundo vemos ejemplos de programación, como los girasoles, el teorema de Fibonacci y las Matryoshkas relacionadas a la recursividad de las cosas. La frase “todos para uno y uno para todos”, que convalida la importancia grupal e individualidad de seres semejantes frente a un problema como los subprogramas y sus mayores como “main” (algoritmo principal).

Mas hoy no es prescindible expulsar lo humano de lo artificial; No olvidemos la importancia del tiempo de reacción de

los programas para el estudio e implementación efectiva de los conocimientos de las ciencias humanas. Las respuestas de algoritmos como Shellsort y Quicksort frente a posibles casos vividos dejan a la imaginación de uno la intriga o curiosidad.

Según Academia Lab (s. f.) Shellsort o método Shell no es un método recursivo, fue elaborado por Donald Shell (1959) el cual clasifica parejas de elementos y los compara buscando la reducción de la amplitud de los pares.

Según Gómez Salgado (2024) Quicksort es un algoritmo fundado por Tony Hoare (1959) basado en la idea de "divide y vencerás"; ordena los valores mediante un punto de referencia o pivote; todos los valores menores a la izquierda y los mayores a la derecha del pivote.

El rendimiento práctico de los algoritmos de ordenación, incluidos Shellsort y Quicksort, varía según el lenguaje, la implementación y el conjunto de datos; por ello, su eficiencia debe evaluarse en función del contexto de uso (Goel et al., 2023).

RESULTADOS

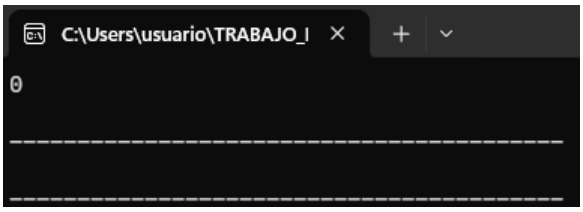
Para el análisis y la obtención de datos, se contemplarán los siguientes tratados:

- "Ti" : Tiempo empleado para un algoritmo.
- "n" : número de elementos de un arreglo.

Consideraremos la aplicación de estos programas de ordenación logarítmica para diferentes arreglos no ordenados y. El tiempo se medirá en segundos (s), se contemplará el tiempo de ejecución de las impresiones por cada algoritmo y "n" constantemente variará de forma creciente.

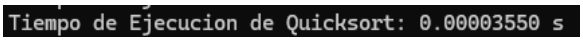
1.1 Zona de Pruebas

1.1.1 Testeo n°01



- Shellsort:

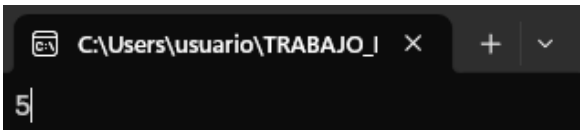




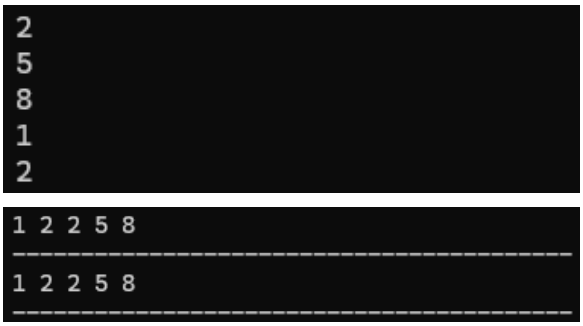


considerando un “n” que adoptará el valor “0” (sin elementos).

1.1.2 Testeo n°02



Fuente: Dev-c++ 5.11

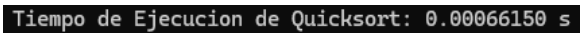


(Impresión del arreglo ordenado de Shellsort y Quicksort)

- Shellsort:



- Quicksort:

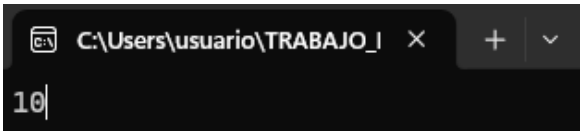


- Tiempo promedio estimado:

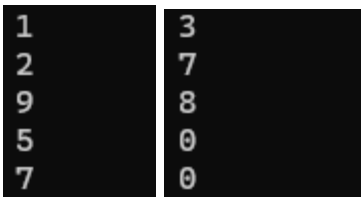


Esta fue la primera recursión (esquema idéntico al anterior). Se compararon ambos programas nuevamente y se obtuvieron los resultados plasmados en las imágenes en las imágenes; considerando un “n” igual al valor numérico “5”.

1.1.3 Testeo n°03



Fuente: Dev-c++ 5.11



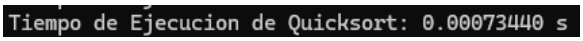


(Impresión del arreglo ordenado de Shellsort y Quicksort)

- Shellsort:



- Quicksort:

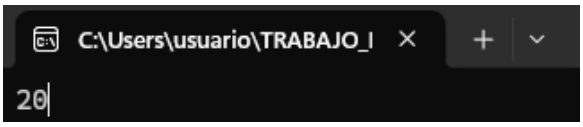


- Tiempo promedio estimado:

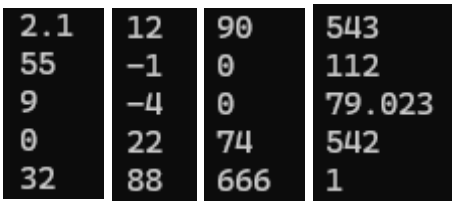


Segunda recursión o práctica experimental, la estructura se mantiene. Se evaluaron los subprogramas de ordenación, retratándose mediante capturas de pantalla; considerando un arreglo de “n=10” elementos.

1.1.4 Testeo n°04



Fuente: Dev-c++ 5.11



(Impresión del arreglo ordenado de Shellsort y Quicksort)

- Shellsort:

```
Tiempo de Ejecucion de Shellsort: 0.00176030 s
```

- Quicksort:

```
Tiempo de Ejecucion de Quicksort: 0.00120290 s
```

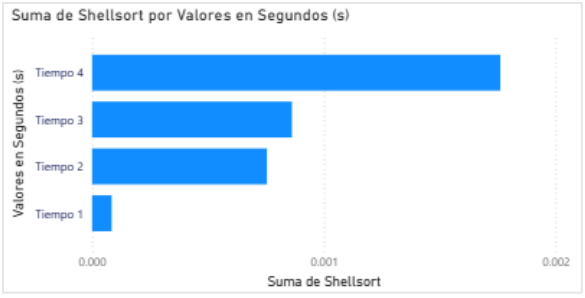
- Tiempo promedio estimado:

```
Tiempo de Ejecucion del Testeo: 0.00148160 s
```

Finalmente, se hace un último llamado experimento cúlmine. Se consideró la misma estructura y se tomaron otras imágenes de datos muestrales. Se analizó el tiempo empleado por ambos algoritmos sobre el caso “n=20” de elementos de un arreglo.

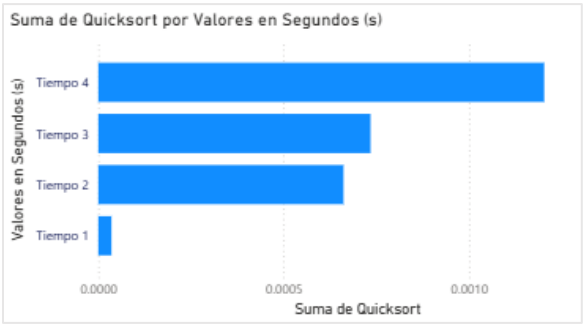
1.2 Representación Gráfica

1.2.1 Gráfica Shellsort:



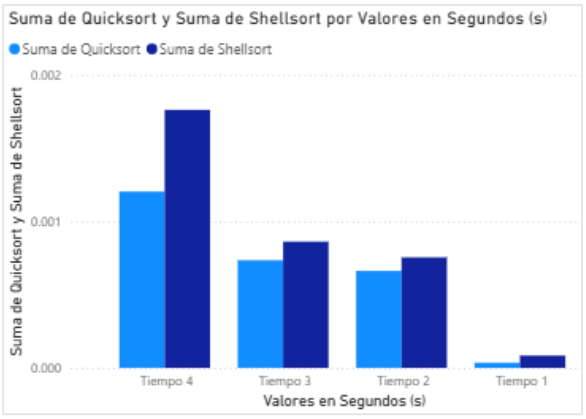
Se tomaron todos los valores de los testeos o las zonas de muestreo con respecto al método Shellsort.

1.2.2 Gráfica Quicksort:



Se plasmaron los valores obtenidos de los periodos de cada experimentación referente al caso base de ordenación Quicksort.

1.2.3 Grafica Comparativa:



Se evidenciaron la comparación de los datos de los tiempos de reacción experimental basados en los métodos Shellsort y Quicksort.

DISCUSIONES

Es cierto que el máximo de elementos es “20” y el mínimo es “0” para todo el sistema de medición de tiempos algorítmicos. Por ende, nuestra población final será el máximo del sistema y también su límite superior; límite inferior igual a “0”. Los tiempos fueron incorporados en una tabla “gráfico de columnas agrupadas” para evidenciar la diferencia en el tiempo de reacción de dos algoritmos: Quicksort y Shellsort. Ambos métodos mostraron tiempos volubles, pero también déficits en el

lapso del procesamiento integrado. Si se convalidan con la ley de la complejidad computacional, adquirirían un campo de análisis estadístico y reflexivo el presente informe.

CONCLUSIONES

- En este informe se ha presentado empíricamente datos resultantes de variables experimentales de cuatro regiones denominados "testeos", incorporándose en estadística descriptiva e inferencial vía ideales metodológicos y valorativos.
- Se visualiza bajo un diferencial integrado entre tiempos muestrales (tabla de mediciones); el promedio de tiempo final de Shellsort y Quicksort fueron: "Ptf.Shell= 0.000880025(s)" y "Ptf.Quick= 0.000634325(s)" respectivamente en todo el sistema (considerar los cuatros tiempos).
- Los algoritmos dependen de su complejidad, eficiencia y estabilidad para cierto caso o problema lógico. Además, ambos programas buscan la rapidez como su tarea principal en diferentes esquemas o estructuras programables.

AGRADECIMIENTOS

Agradezco al profesor Hugo Barraza por ser el motivo de la corriente de pensamientos, subjetividad y empirismo en ámbitos de la programación avanzada de los estudiantes de primer año de la carrera profesional de Ingeniería en Informática y Sistemas.

REFERENCIAS

- ✓ Academia Lab. (s. f.). *Shellsort*. https://academia-lab.com/enciclopedia/shellsort/
- ✓ Goel, K., Dwivedi, P., & Sharma, O. (2023). Performance analysis of various sorting algorithms: Comparison and optimization. En *2023 11th International Conference on Intelligent Systems and Embedded Design (ISED)* (pp. 1–5). IEEE. https://doi.org/10.1109/ISED59382.2023.10444609
- ✓ Ciencia LR. (2024, 19 de septiembre). *El teorema no era de Pitágoras: tablilla de arcilla revela que existía 1000 años antes del nacimiento del matemático*. La República. https://larepublica.pe/ciencia/202

4/09/09/el-teorema-no-era-de-pitagoras-una-tablilla-de-arcilla-revela-que-ya-existia-1000-anos-antes-del-matematico-griego-845361

- ✓ LaHistoria. (s. f.). *Historia de los algoritmos*. https://lahistoria.info/historia-de-los-algoritmos/
- ✓ Gómez Salgado, L. (2024, 14 de octubre). *Quicksort: el algoritmo de ordenación que todo programador debe conocer*. KeepCoding. https://keepcoding.io/blog/que-es-quicksort-y-como-funciona-programacion/